FIAP · PÓS-TECH SOFTWARE ARCHITECTURE (SOAT)

Tech Challenge — Fase 4

fiap-officine: gestão de oficina mecânica em microsserviços com Saga orquestrada

Grupo [Nº DO GRUPO]

Participante	RM
[NOME COMPLETO]	[RM]
[NOME COMPLETO]	[RM]
[NOME COMPLETO]	[RM]
[NOME COMPLETO]	[RM]
[NOME COMPLETO]	[RM]

Novembro de 2026

1. Entregáveis

Vídeo de demonstração

[LINK DO VÍDEO — YouTube não listado ou Google Drive]

Repositórios (organização `fiap-officine` no GitHub)

Repositório	Conteúdo
github.com/fiap-officine/fiap-officine-api	**OS Service**: orquestrador da saga, clientes, veículos, catálogo e ordens de serviço (PostgreSQL `os_db`)
github.com/fiap-officine/fiap-officine-billing-service	**Billing Service**: orçamentos, aprovação e pagamentos no Mercado Pago (PostgreSQL `billing_db`)
github.com/fiap-officine/fiap-officine-execution-service	**Execution Service**: estoque, reserva de peças e fila de execução (DynamoDB)
github.com/fiap-officine/fiap-officine-lambda	Lambda de autenticação do cliente por CPF e authorizer JWT do API Gateway
github.com/fiap-officine/fiap-officine-kubernets	Plataforma: VPC, cluster K3s, API Gateway, ECR (Terraform)
github.com/fiap-officine/fiap-officine-database	Instância RDS PostgreSQL compartilhada (Terraform)
github.com/fiap-officine/fiap-officine-e2e	Ambiente ponta a ponta (docker-compose), roteiro automatizado da saga e collection Postman

Conteúdo deste documento

1. Entregáveis
2. Arquitetura geral
3. Divisão dos microsserviços
4. Decisões tecnológicas
5. Saga orquestrada
6. Comunicação e contratos
7. Integração com o Mercado Pago
8. Qualidade e testes
9. CI/CD e proteção de branches
10. Infraestrutura e Kubernetes
11. Observabilidade
12. Trade-offs e limitações

2. Arquitetura geral

O monólito da Fase 3 foi dividido em **três microsserviços**, cada um com repositório, pipeline, infraestrutura (Terraform) e banco de dados próprios. O cliente e o operador falam com os serviços por **REST**, através do API Gateway e do Ingress Traefik; entre si, os serviços conversam apenas por **mensageria (Amazon SQS)**. Nenhum serviço acessa o banco de outro — inclusive a Lambda de autenticação, que passou a consultar o OS Service por uma rota interna.

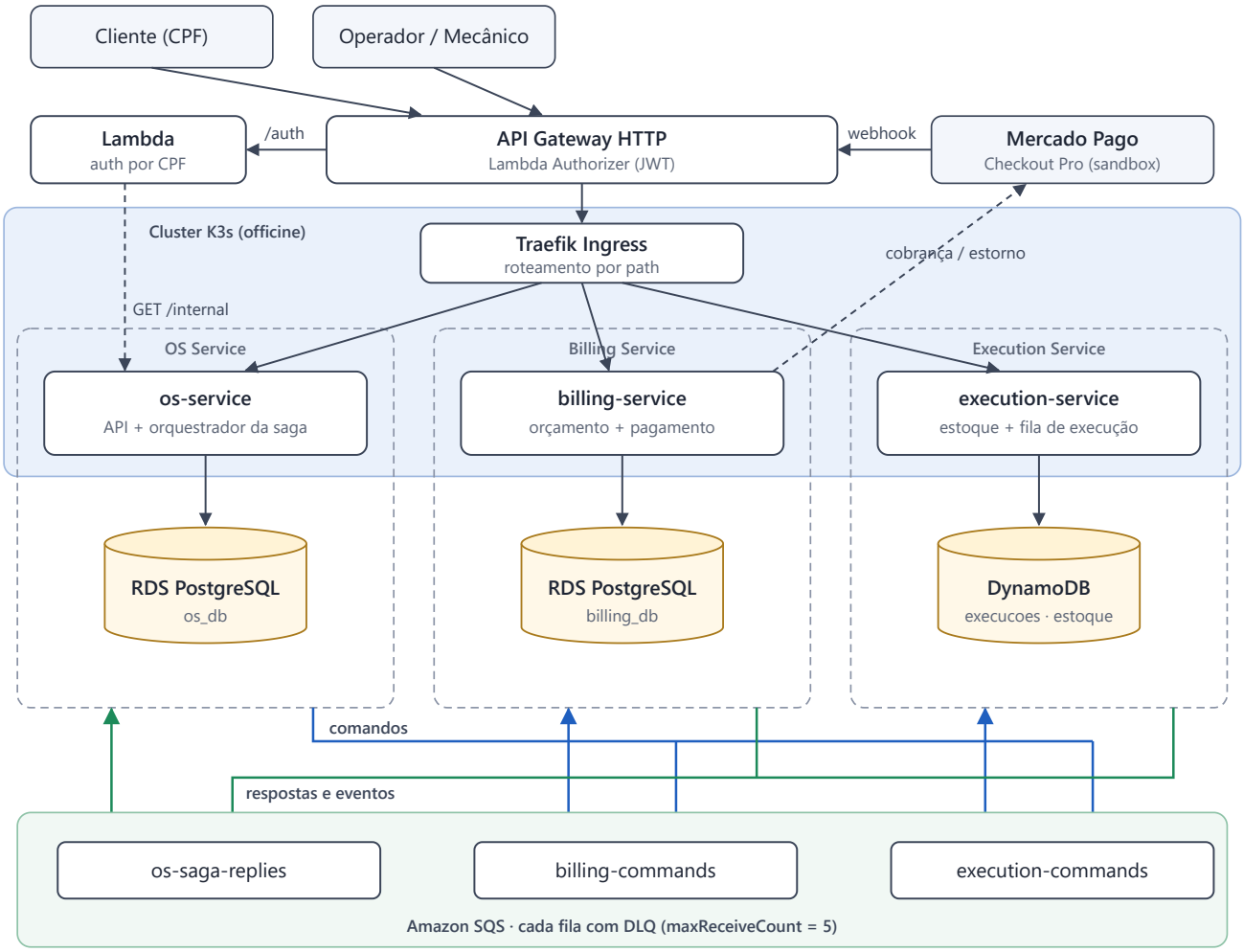


Figura 1 — Visão geral: REST na borda, SQS entre os serviços, um banco por serviço. Os três serviços enviam APM, traces e logs ao New Relic.

Cada serviço roda em um único pod com a API, o consumer da fila, o *outbox relay* e os varredores (timeouts da saga e reconciliação de pagamentos) no mesmo processo, para caber no nó K3s `t3.micro`.

3. Divisão dos microsserviços

Serviço	Responsabilidades	Dados que possui	Banco
OS Service (orquestrador)	Clientes, veículos, catálogo (serviços e peças com preço), abertura e acompanhamento da OS, histórico, **orquestração da saga**, cancelamento e entrega, login do operador	`clientes`, `veiculos`, `servicos`, `pecas`, `ordens_servico` e itens, `historico`, `saga_instances`, `saga_log`, `outbox`, `processed_messages`	PostgreSQL `os_db`
Billing Service	Gerar orçamento, aprovação/recusa pelo cliente, cobrança no Mercado Pago, webhook, estorno (compensação), reconciliação de pagamentos	`orcamentos`, `orcamento_itens`, `pagamentos`, `outbox`, `processed_messages`	PostgreSQL `billing_db`
Execution Service	Saldo e reserva de estoque, fila priorizada de execução, diagnóstico, reparo e finalização pelo mecânico	Tabela `execucoes` (fila, linha do tempo, idempotência e outbox) e tabela `estoque`	DynamoDB

Critérios da divisão

- **Capacidades de negócio distintas**: atendimento (OS), financeiro (orçamento e pagamento) e produção (oficina e estoque) mudam por motivos diferentes e têm atores diferentes (atendente, cliente, mecânico).
- **O estoque foi para o Execution**: a reserva de peças é um passo da saga com compensação natural (liberar a reserva) e quem consome peça é a produção. O **catálogo** (nome e preço) ficou no OS Service: o preço é "congelado" no comando `GerarOrcamento`, sem chamada síncrona entre serviços.
- **Banco por serviço**: OS e Billing precisam de transações ACID (saga, outbox e pagamento na mesma transação); o Execution precisa de reserva atômica e condicional, o que o DynamoDB oferece com `TransactWriteItems`.

> **Isolamento dos bancos SQL.** O Free Tier cobre uma instância RDS. Cada serviço tem **banco lógico e usuário próprios**, dono do seu banco, sem GRANT cruzado e com `CONNECT` revogado de `PUBLIC`. A criação é feita por um Job Kubernetes idempotente no pipeline de cada serviço (o RDS não é público). Uma segunda instância eliminaria o compartilhamento físico, se houver orçamento.

4. Decisões tecnológicas

Decisão	Escolha	Alternativas	Justificativa
Linguagem	Python 3.12, FastAPI, SQLAlchemy 2, uv	Java/Spring, Node	Domínio do time; reaproveita middleware, logs JSON e testes da Fase 3
Broker	Amazon SQS (standard + DLQ)	RabbitMQ no cluster, Kafka	Gerenciado, DLQ nativa, sem ocupar RAM do nó K3s `t3.micro`, custo dentro do Free Tier
NoSQL	DynamoDB	MongoDB	`TransactWriteItems` condicional dá reserva atômica e idempotente; modelo chave-valor serve para a fila e a linha do tempo
SQL	PostgreSQL 16 (RDS)	MySQL	Já provisionado; ACID para OS, saga, outbox e pagamentos
Saga	Orquestrada	Coreografada	Ver seção 5
Pagamentos	Mercado Pago Checkout Pro (SDK oficial)	Pix direto, Stripe	Requisito do desafio; sandbox com compradores e cartões de teste; API de estorno
Testes	pytest, pytest-bdd, moto	behave, LocalStack nos testes	BDD integrado ao pytest e à cobertura; `moto` simula SQS e DynamoDB sem rede
Qualidade	SonarCloud	SonarQube self-hosted	Gratuito para repositórios públicos, Quality Gate no PR
Deploy	GitHub Actions, ECR, Kustomize	Helm, ArgoCD	Pipeline independente por repositório; Kustomize sem dependências extras
Observabilidade	New Relic (Fase 3)	Prometheus, Grafana, Jaeger	Já implementado; o desafio pede reaproveitar o monitoramento

5. Saga orquestrada

Por que orquestrada

1. **O fluxo é linear e tem esperas longas** (aprovação do cliente e pagamento). Um orquestrador com estado persistido (`saga_instances`) trata prazos e retomadas de forma explícita.
2. **A OS é o agregado dono do processo**: status da OS e estado da saga ficam no mesmo banco e na mesma transação.
3. **Compensações centralizadas e legíveis**: a ordem do rollback está em um único lugar (uma tabela de transições declarativa).
4. **Visibilidade**: `GET /api/v1/ordens-servico/{id}/saga` mostra o passo atual e a linha do tempo de cada OS.
5. **Sem dependências cíclicas**: Billing e Execution não se conhecem; só respondem comandos.

Trade-offs aceitos: o orquestrador concentra a lógica do fluxo (mitigado por ser *stateless*, com estado no banco) e o OS Service conhece os contratos dos outros serviços.

Passos, comandos e compensações

#	Passo (participante)	Comando	Sucesso	Falha	Compensação
T1	Criar OS (OS Service)	—	—	validação 4xx	cancelar OS
T2	Gerar orçamento (Billing)	`GerarOrcamento`	`OrcamentoGerado`	`OrcamentoFalhou`	`CancelarOrcamento`
T3	Aprovação do cliente (Billing, REST)	—	`OrcamentoAprovado`	`OrcamentoRecusado` / prazo	—
T4	Pagamento (Billing + Mercado Pago)	`SolicitarPagamento`	`PagamentoConfirmado`	`PagamentoRecusado` / prazo	`EstornarPagamento`
T5	Reservar peças e enfileirar (Execution)	`ReservarEEnfileirar`	`ExecucaoAgendada`	`ReservaFalhou`	`LiberarReserva`
T6	Diagnóstico, reparo, finalização	—	`DiagnosticoIniciado` , `ReparoIniciado` , `ExecucaoFinalizada`	—	**pivô**: sem compensação

Ordem das compensações

Falha em	Compensações (em ordem inversa, cada uma confirmada antes da próxima)
T2 orçamento falhou	cancelar OS
T3 recusa ou prazo de aprovação	`CancelarOrcamento` → cancelar OS
T4 recusa ou prazo de pagamento	`CancelarOrcamento` → cancelar OS
T5 estoque insuficiente	`EstornarPagamento` → `CancelarOrcamento` → cancelar OS
Cancelamento com a OS na fila	`LiberarReserva` → `EstornarPagamento` → `CancelarOrcamento` → cancelar OS

Máquina de estados da saga

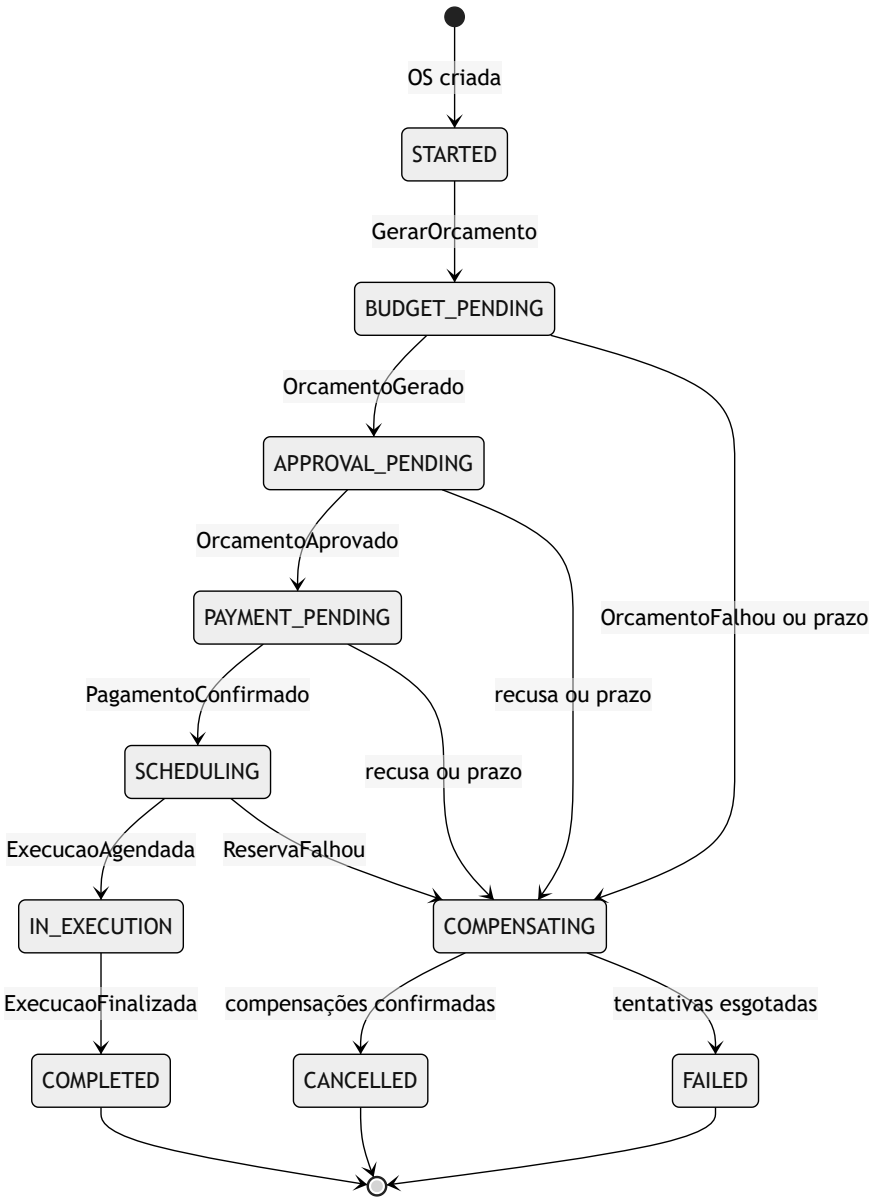
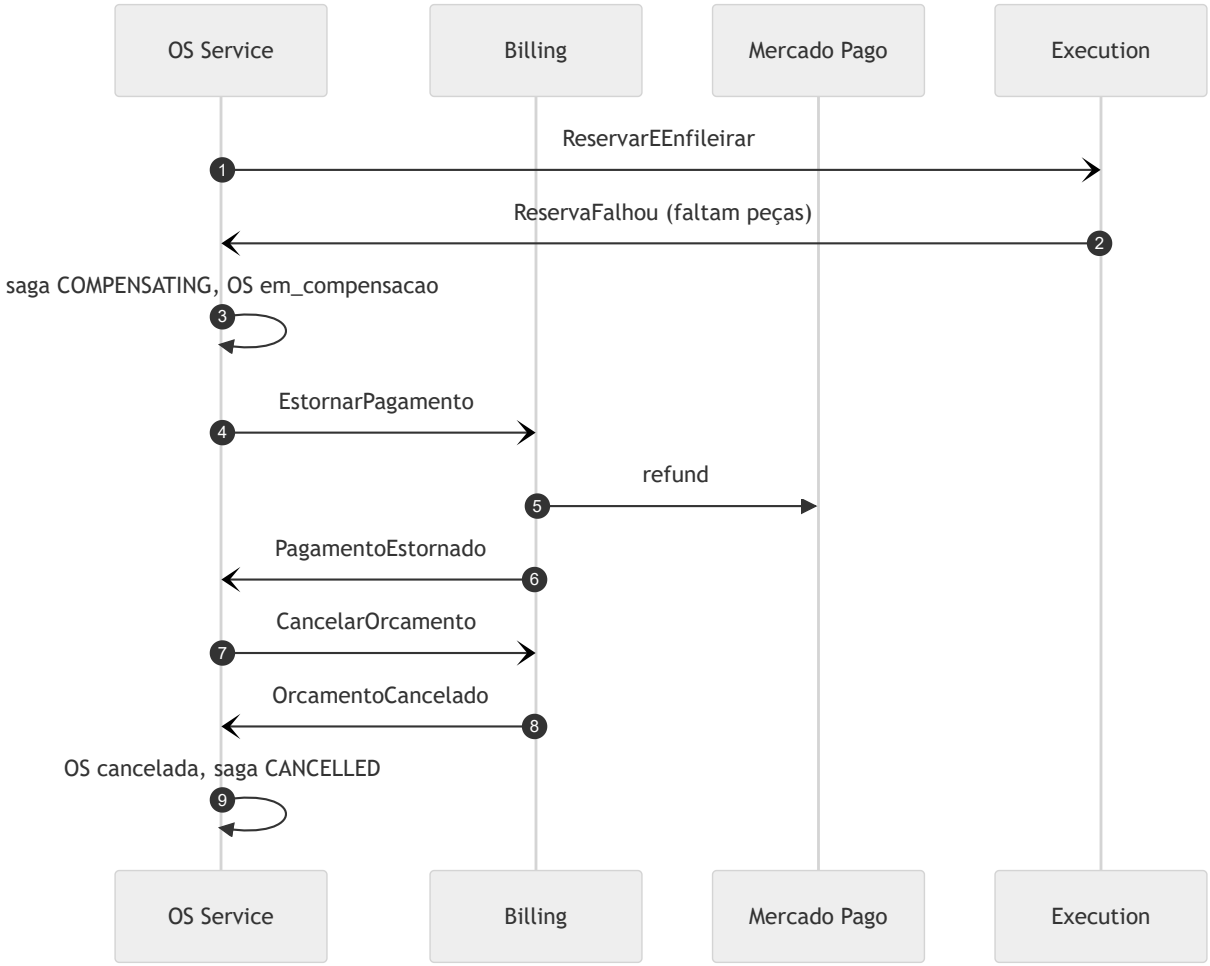


Figura 2 — Estados da saga (tabela `saga_instances`). Status da OS: recebida → aguardando_orcamento → aguardando_aprovacao → aguardando_pagamento → aguardando_execucao → em_diagnostico → em_execucao → finalizada → entregue; antes do pivô, em_compensacao → cancelada.

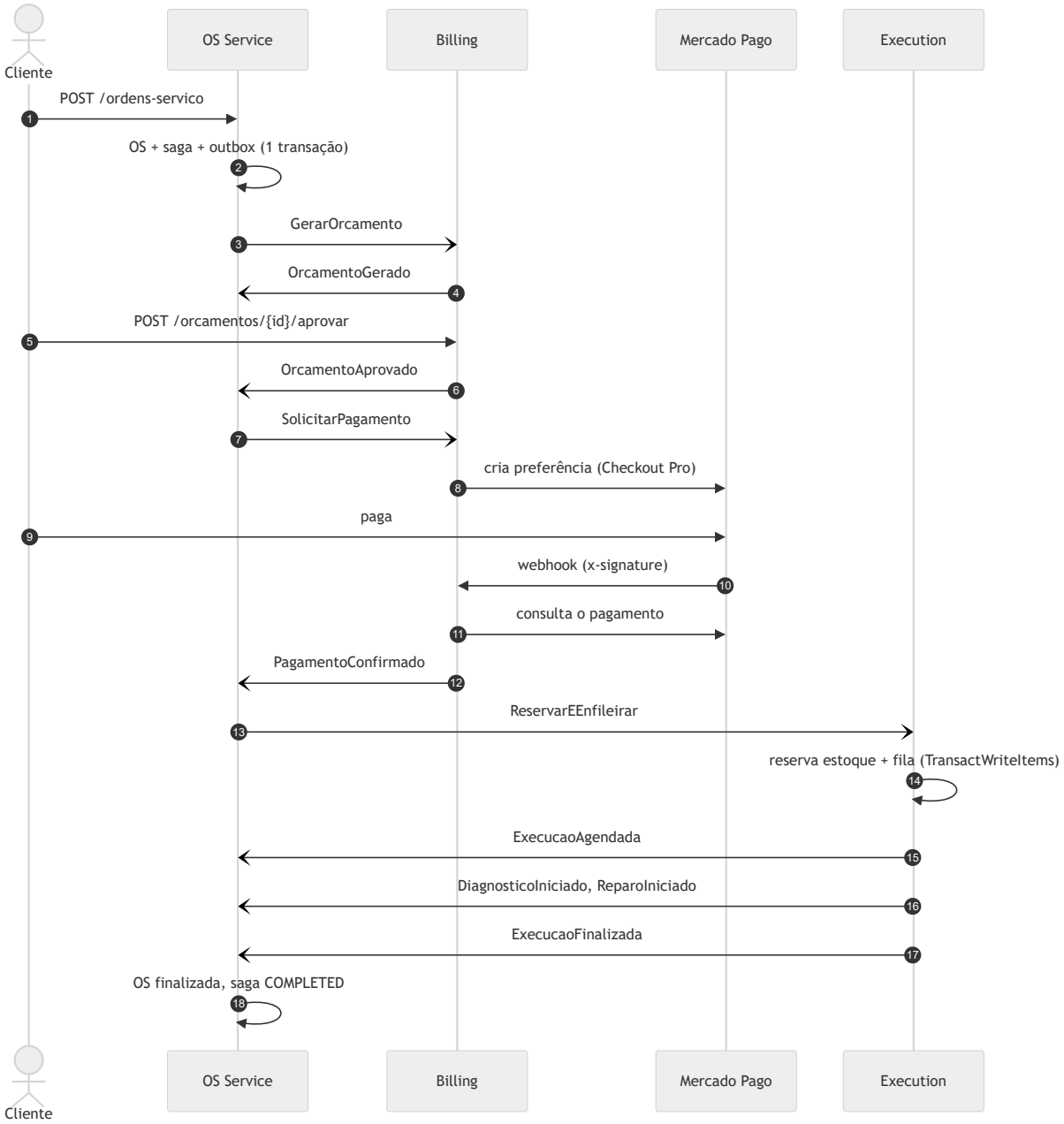
Garantias técnicas

- **Transactional Outbox**: comando/evento gravado na tabela `outbox` na mesma transação da mudança de estado (no DynamoDB, como item na mesma `TransactWriteItems`); um *relay* publica no SQS.
- **Consumidores idempotentes**: `processed_messages` (PostgreSQL) ou item condicional `attribute_not_exists` (DynamoDB) na mesma transação do efeito — o SQS entrega *at-least-once*.
- **Retries e DLQ**: `maxReceiveCount = 5` em todas as filas; alerta quando a DLQ recebe mensagens. Compensações também são repetidas; ao esgotar as tentativas a saga vai para `FAILED` e dispara alerta.
- **Prazos**: `deadline_at` por passo (orçamento 5 min, aprovação 48 h, pagamento 30 min — 2 min em homolog para a demonstração) e um varredor periódico que dispara a compensação.
- **Concorrência**: `SELECT … FOR UPDATE` na linha da saga ao processar cada resposta.
- **Mensagens fora de ordem ou atrasadas**: eventos de progresso fora de ordem são aceitos; um pagamento que chega depois do cancelamento é estornado automaticamente.
- **Pivô explícito**: depois de `ReparoIniciado` não há compensação; o cancelamento responde 409.

Sequência — compensação por estoque insuficiente



Sequência — caminho feliz



6. Comunicação e contratos

Tipo	Onde é usado
REST síncrono	Cliente e operador → serviços (via API Gateway e Ingress); Lambda → OS Service (`GET /internal/clientes/status/{cpf}` , protegido por token interno); Billing → API do Mercado Pago
Mensageria assíncrona (SQS)	Toda a orquestração da saga: comandos do OS Service e respostas/eventos dos participantes

Fila (+ DLQ)	Produtor	Consumidor	Mensagens
`billing-commands`	OS Service	Billing	`GerarOrcamento` , `SolicitarPagamento` , `EstornarPagamento` , `CancelarOrcamento`
`execution-commands`	OS Service	Execution	`ReservarEEnfileirar` , `LiberarReserva`
`os-saga-replies`	Billing, Execution	OS Service	respostas e eventos de progresso

Cada fila pertence ao repositório do serviço que a consome (Terraform em `infra/`), com long polling e DLQ.

Envelope padrão

```
{
  "message_id": "uuid",            // chave de idempotência
  "type": "GerarOrcamento",
  "kind": "command | reply | event",
  "saga_id": "uuid", "os_id": 123, "correlation_id": "uuid",
  "occurred_at": "2026-11-07T17:00:00Z", "source": "os-service", "version": 1,
  "trace": { "newrelic": "...", "traceparent": "...", "tracestate": "..." },
  "payload": { }
}
```

Os contratos (JSON Schema do envelope e payloads de cada mensagem) estão versionados em `docs/contracts/` de cada serviço, e as APIs REST em `docs/openapi.json` (Swagger em `/docs` , `/billing/docs` e `/execution/docs`).

7. Integração com o Mercado Pago

- **Cobrança**: ao receber `SolicitarPagamento`, o Billing cria uma preferência do **Checkout Pro** (SDK oficial) com `external_reference` = id do pagamento e devolve o link de checkout ao cliente.
- **Webhook**: `POST /api/v1/pagamentos/webhook`, rota pública no API Gateway. A assinatura `x-signature` (HMAC-SHA256 sobre `id`, `request-id` e `ts`) é validada e o pagamento é **reconsultado na API do Mercado Pago** antes de qualquer mudança — o corpo da notificação nunca é confiado.
- **Estorno como compensação**: `EstornarPagamento` chama a API de refund; também estorna automaticamente um pagamento que chegue depois de a OS ter sido cancelada.
- **Reconciliação**: se o webhook não chegar, um job consulta periodicamente os pagamentos pendentes por `external_reference`.
- **Desenvolvimento e testes**: a porta `PaymentGateway` tem duas implementações — `MercadoPagoGateway` (sandbox/produção) e `FakeGateway` (local, testes e roteiro E2E).

[INSERIR PRINT: checkout do Mercado Pago no sandbox e/ou pagamento aprovado e estornado no painel]

8. Qualidade e testes

Repositório	Testes	Cobertura (branch)	Destaques
OS Service	208	94,7%	orquestrador (todos os caminhos e compensações), mensageria, API, 6 cenários BDD
Billing Service	57	93,3%	comandos da saga, assinatura do webhook, gateway, reconciliação
Execution Service	39	93,2%	reserva condicional no DynamoDB (moto), fila, idempotência
Lambda Auth	27	94%	authorizer JWT e cliente HTTP do OS Service

O gate `--cov-fail-under=80` roda em todo PR e push, sem exclusões no relatório de cobertura. O SonarCloud recebe o `coverage.xml` e o job `quality` só passa com o Quality Gate aprovado.

BDD — fluxo completo da saga (pytest-bdd, Gherkin em português)

`tests/bdd/features/saga_ordem_servico.feature` executa o orquestrador real com participantes simulados trocando mensagens pelo SQS (mockado com `moto`):

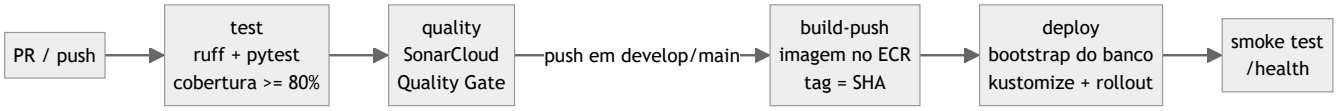
- Fluxo completo até a OS finalizada e a saga `COMPLETED`
- Orçamento recusado → orçamento cancelado e OS cancelada
- Estoque insuficiente após o pagamento → estorno e cancelamento
- Prazo de pagamento vencido → compensação
- Cancelamento com a OS na fila → liberação da reserva, estorno e cancelamento
- Mensagem duplicada não gera efeito em dobro

Além disso, o repositório `fiap-officine-e2e` sobe os três serviços com PostgreSQL e LocalStack e roda o caminho feliz e a compensação de ponta a ponta.

[INSERIR PRINT: SonarCloud — cobertura e Quality Gate dos três serviços]

9. CI/CD e proteção de branches

Cada microsserviço tem pipeline próprio em GitHub Actions (`.github/workflows/ci-cd.yml`):



- **Infraestrutura do serviço** (`infra.yml`): Terraform com state remoto no S3 — `plan` no PR, `apply` no merge (filas + DLQ, tabelas DynamoDB, políticas IAM).
- **Segredos**: nunca versionados; gerados no pipeline a partir de GitHub Secrets como Secrets do Kubernetes.
- **Imagem imutável**: a tag é o SHA do commit, aplicada com `kustomize edit set image` ; o job só termina quando o `rollout status` e o smoke test passam.
- **Branches protegidas**: `main` e `develop` exigem Pull Request, aprovação e os checks `test` e `quality` ; push direto e force push bloqueados.

[INSERIR PRINT: ruleset da branch main (Settings → Rules) e um PR com os checks aprovados]

[INSERIR PRINT: execução do pipeline com build, push no ECR e deploy]

10. Infraestrutura e Kubernetes

- **Plataforma** (`fiap-officine-kubernets`): VPC, cluster K3s, API Gateway HTTP com Lambda Authorizer e um repositório ECR por serviço. **Banco** (`fiap-officine-database`): instância RDS PostgreSQL.
- **Por serviço** (`infra/`): filas e DLQ, tabelas DynamoDB (com GSI da fila, TTL e PITR) e política IAM.
- **Manifests** (`k8s/` , Kustomize): base com Deployment (initContainer de migração Alembic, probes de readiness e liveness, limites de recursos, usuário não-root), Service, HPA e Ingress Traefik com roteamento por path; overlay de homolog com as configurações do ambiente. Namespace único `officine` .
- **Acesso à AWS pelos pods**: instance profile do nó (IMDSv2 com hop limit 2); cada serviço anexa sua política.
- **Ambiente local**: `docker-compose` em cada repositório e no `fiap-officine-e2e` , com LocalStack (SQS e DynamoDB) e PostgreSQL.

[INSERIR PRINT: `kubectl -n officine get pods,svc,ingress,hpa` com os três serviços]

11. Observabilidade

- **APM por serviço**: `officine-os-service`, `officine-billing-service` e `officine-execution-service` aparecem separados no Service Map.
- **Distributed tracing atravessando o SQS**: os headers de trace vão no campo `trace` do envelope e são aceitos pelo consumer (`transport_type="Queue"`) — um único trace mostra OS → Billing → OS → Execution.
- **Logs estruturados** (JSON) com `correlation_id`, `saga_id` e `os_id`, encaminhados ao New Relic como atributos: `context.saga_id:"…"` mostra a saga inteira nos três serviços.
- **Eventos de negócio**: `SagaStepEvent`, `SagaCompensationEvent`, `SagaFailedEvent`, `PagamentoEvent`, `ExecucaoEvent`, `MessageProcessingFailure`, `WebhookRejected`.
- **Dashboard e alertas como código** (Terraform, `observability/newrelic/`): painel "Saga de OS" (funil, tempo por passo, compensações, saúde dos serviços) e alertas de saga `FAILED`, DLQ com mensagens, falhas no consumer, latência p95, taxa de erro e webhook rejeitado.

[INSERIR PRINT: distributed trace atravessando os três serviços]

[INSERIR PRINT: dashboard "fiap-officine — Saga de OS"]

[INSERIR PRINT: logs filtrados por `context.saga_id` e política de alertas]

12. Trade-offs e limitações

Decisão	Motivo e consequência
Ambiente único (homolog)	Projeto acadêmico: `develop` e `main` publicam no mesmo cluster. A estrutura de overlays e tfvars permite acrescentar produção sem mudar os serviços.
Uma instância RDS para dois bancos	Free Tier. Isolamento lógico (banco e usuário por serviço, sem GRANT cruzado), não físico.
Consumer no mesmo processo da API	Cabe no nó `t3.micro` (um pod por serviço). Em escala, separar em Deployments de worker.
Pods usam a role do nó	No K3s não há IRSA; todos os pods compartilham o instance profile. No EKS, trocar por IRSA (privilégio mínimo por serviço).
SQS standard	Sem garantia de ordem: o orquestrador tolera eventos fora de ordem e duplicados (idempotência).
Itens da OS definidos na abertura	O orçamento é gerado no início da saga; mudanças de escopo exigem nova OS.
Bancos criados por Job no cluster	O RDS é privado; o pipeline não alcança o banco. Um Job idempotente cria banco e usuário a cada deploy.

As decisões estão registradas nos ADRs do OS Service (`docs/architecture/adr/`), em especial o ADR-006 (microsserviços e saga orquestrada) e o ADR-007 (SQS, DynamoDB e banco por serviço).